# kwirth

## Technical Overview

Kubernetes observability & operations

**Architecture** · **Deployment** · **Integration** · **Security**

Open source · Apache License 2.0 · github.com/kwirthmagnify/kwirth

OVERVIEW

# What is Kwirth?

Kwirth replaces a full observability stack (Prometheus + Grafana + Loki + Alertmanager + …) with a single pod. Real-time logs, metrics, alerting, security scanning, BYO-LLM AI analysis and a complete Kubernetes manager — all in one pod.

### One pod, zero deps

Single container. State in Kubernetes — ConfigMaps, Secrets, a PVC or files. No external DB, queue or sidecar.

### Real-time first

All data flows over WebSockets — no polling. Logs, metrics, alerts and AI answers arrive as they happen.

### Multi-cluster native

Connect many clusters from one instance, each with independent access control. Federated in the front end.

### Extensible by design

Hot-reloadable plugins, providers, senders, themes, homepages, IdPs, logins and webhooks — no pod restart.

### Secure by default

Fine-grained RBAC with per-plugin scopes, SSO via external IdPs, and permanent/volatile/bearer access keys.

### Clean stack

TypeScript end-to-end. Node.js + Express back end, React + MUI front end, direct Kubernetes API access.

Listens on port 3883 · in-cluster or desktop (Kwirth Magnify) · started 2022

PART 1

# Architecture

ARCHITECTURE

# The single-pod topology

ARCHITECTURE

# Inside the pod

## Core — Frontend

React + MUI single-page app, served as static assets by the Express server. Renders channels, dashboards and the extension managers. Talks to the back end over REST + WebSocket.

## Core — Backend

Node.js + Express. Exposes the REST API and the WebSocket server, hosts the channel/provider/sender subsystems, and reaches clusters directly through @kubernetes/client-node — no middleman.

## State

Persisted in Kubernetes-native stores: ConfigMaps, Secrets, a PVC, or plain files — users, API keys, extension config and workspaces. No external database or message queue.

## Real-time transport

A single multiplexed WebSocket carries every stream — logs, metrics, events, AI. Service instance ids let multiple instances of a service share the socket. No polling anywhere.

ARCHITECTURE

# Extensibility — families & areas

Everything above the core is a hot-reloadable extension, installed / configured / enabled from the UI with no pod restart. Grouped in four runtime areas plus packaging.

### UI

Themes — repaint the whole shell.
Homepages — swap the landing dashboard.

### Security

Identity providers — SSO connectors.
Logins — pluggable sign-in methods.

### Data

Plugins — the channels users work in.
Providers — feed data into channels.

### Integration

Senders — outbound delivery.
Webhooks — inbound HTTP callbacks.

**Packaging:** Packs bundle many extensions in one .tgz · Documentation packages are docsify sites Kwirth serves itself.

**Channels (13):** Log · Metrics* · Alert · Ops · Fileman · Trivy · Magnify* · Topology · Pinocchio · Censor · mIRC · News · Echo (*built-in)

**Providers:** Events* · Metrics · Business (built-in) · Kafka · OpenTelemetry · Syslog · Trivy · Tick · Validating (installable)

**Senders:** console · file · email-smtp · email-resend · teams + pipelines: tee · regex · timed · ratelimit · composite

PART 2

# Deployment

DEPLOYMENT

# Five ways to run it

### ① Kubernetes — Helm (recommended)

helm repo add + helm install kwirth kwirth/kwirth -n kwirth --create-namespace. Tune with values.yaml.

### ② Kubernetes — manifests

kubectl apply -f .../test/kwirth.yaml for an express setup; edit the YAML to change defaults.

### ③ Docker

docker run -p 3883:3883 with your kubeconfig mounted so Kwirth can reach the cluster.

### ④ External — no Docker

npm i -g @kwirthmagnify/kwirth-external, then kwirth-external start --front --port ...

### ⑤ Desktop — Kwirth Magnify

Native installer for Windows, macOS and Linux. A context selector picks the cluster: LOCAL (kubeconfig contexts) or REMOTE (through a Kwirth server). The source cluster appears as inDesktop.

DEPLOYMENT

# Helm — install & configure

```
helm repo add kwirth \
  https://github.com/kwirthmagnify/kwirth

helm install kwirth kwirth/kwirth \
  -n kwirth --create-namespace \
  -f values.yaml
```

```
kwirth:
  config:
    channelMetrics: "true"
    channelMagnify: "true"
    rootpath: /kwirth
  masterkey: <change-me>
  image: kwirthmagnify/kwirth:0.5.287
```

## Most useful Helm options

| option | meaning |
|---|---|
| `masterkey` | signs the access keys issued to clients — change it |
| `rootpath` | path Kwirth is served under (default /kwirth) |
| `image` | full image ref (default .../kwirth:latest) |
| `resources` | pod CPU / memory (K8s format) |
| `ingress.*` | deploy an Ingress (nginx / agic, hostname) |
| `nginx.tls` | enable TLS + secret holding CRT + KEY |

*Channels are plugins — install Log, Ops, Trivy, Fileman... from the plugin management UI, not via Helm flags.*

DEPLOYMENT

# Networking & operations

- Port — Kwirth listens on 3883 inside the container; publish/map it as you like for Docker or external runs.
- Sub-path — serve under a custom path by creating an Ingress and setting the ROOTPATH env var to the same path (e.g. /kwirth). That is the one thing Kwirth must know.
- TLS — terminate at the Ingress (nginx.tls + a secret with CRT + KEY) or in front of the pod.
- Namespace — fully selectable; the Helm chart creates it (kwirth by default).
- Self-update — restart Kwirth from the UI; with the latest image tag this pulls and runs the newest version.
- Resources — sensible defaults (limits cpu 1 / mem 2Gi, requests mem 256Mi); override per environment.

```
env:
  - name: ROOTPATH
    value: "/kwirth"

# → http://host/kwirth
```

## Consumers

External tools — Backstage, Grafana, CI/CD — consume Kwirth through its REST / WebSocket API.

PART 3

# Security

SECURITY

# The authorization model

Every request is authorized against an access key that encodes exactly which cluster resources — namespaces, workloads, pods, containers — and which actions the holder may reach. Security works at three levels.

### Administrator

The built-in admin account manages users, API keys and identity providers. Only the admin scope may perform security-related actions.

### User

Created with a fine-grained set of resources and scopes. Scopes are declared dynamically by each plugin and served by the core — installing a channel can add new permissions.

### API

Access keys let external apps and other Kwirth instances reach resources across clusters, carrying their own scopes, target resources and optional lease/expiry.

*Fine-grained RBAC · per-plugin dynamic scopes · enforced in the back end — never trusting the client.*

SECURITY

# Authentication & Single Sign-On

- Built-in user / password — always available for the bootstrap admin account.
- Login extensions — pluggable sign-in methods (e.g. anonymous), each with its own config schema.
- SSO via external Identity Providers — can be combined with the above.

## SSO flow — hardened

- OIDC / OAuth2 connectors.
- PKCE + state, back-channel token exchange — the browser never sees provider tokens.
- Single-use login hand-off and anti open-redirect checks.
- No auto-provisioning — the user must already exist and be bound to the IdP they use.

## Bundled connectors (installable)

Google
Gmail and Google Workspace accounts.

GitLab
GitLab.com and self-managed (on-prem).

GitHub
GitHub.com and GitHub Enterprise Server.

Connectors are extensions, so more providers — including third-party ones — can be added over time.

SECURITY

# Access keys & inbound webhooks

### permanent

Persisted in the cluster. Long-lived keys for other Kwirth instances and external integrations.

### volatile

In-memory only. Disappear when the pod restarts — ideal for short-lived, session-scoped access.

### bearer

Signed and handed to clients at login. Verified with the masterkey; can carry a lease / expiry.

**masterkey —** signs and verifies every issued access key. Change it from the default before production.

**Inbound webhooks —** endpoints where external systems POST events into Kwirth are protected by the same access-key model, so only authorized callers get through. Each config exposes a tokenized URL that can be rotated.

PART 4

# Integration

INTEGRATION

# Inbound, outbound & consumers

Kwirth plugs into your ecosystem in every direction — ingesting external data, receiving inbound calls, pushing notifications out, and exposing everything through its API.

### Ingest — Providers

Feed external data into channels.

Built-in: Events, Metrics, Business.

Installable: Kafka, OpenTelemetry (OTLP/HTTP), Syslog, Trivy, Tick, Validating.

### Inbound — Webhooks

External systems POST events to a tokenized, rotatable URL. Kwirth parses the payload and routes it to a target channel or plugin.

### Outbound — Senders

Push alerts and notifications out: console, file, email (SMTP / Resend), Microsoft Teams. Pinocchio findings delivered via senders.

### Consume — API

External tools pull from the REST / WebSocket API — Backstage, Grafana, CI/CD, other Kwirth instances — shared via API keys.

*Every integration point authorizes against the same access-key model.*

INTEGRATION

# Routing pipelines & webhook flow

## Sender pipelines — the dispatcher pattern

Routing senders intercept a message and decide the downstream target — and they chain freely.

- tee — fan out to many senders at once.
- regex — route or drop by rule; first match wins.
- timed — gate by time-of-day / day-of-week (timezone-aware).
- ratelimit — throttle to a max rate so alert storms cannot flood a destination.
- composite — visual pipeline designer in the UI (no JSON editing).

```
regex -> timed ->  tee  ->  email-smtp + file
```

## Inbound webhook flow

```
external service --POST--> webhook URL (token)
     -->  parse  -->  target channel / plugin
```

- Each webhook holds named configurations.
- Every config exposes a unique URL carrying a secret token — rotatable.
- The target is one of your installed channels / plugins.
- Fields come from the webhook's own config schema — typed, no guesswork.
- Protected by the access-key model — only authorized callers get through.

In short

# One pod. Real-time. Multi-cluster. Extensible. Secure.

- Architecture — one container (React + Node/Express), direct Kubernetes API, real-time over WebSocket, state in ConfigMaps / Secrets / PVC / files.
- Deployment — Helm, manifests, Docker, external Node package or native desktop; port 3883, ROOTPATH for sub-paths, self-update, channels as plugins.
- Integration — ingest via providers (Kafka, OTLP, Syslog…), inbound webhooks, outbound senders with routing pipelines, and a REST / WebSocket API for Backstage, Grafana & CI/CD.
- Security — three-level RBAC with per-plugin scopes, SSO via OIDC/OAuth2 (PKCE + back-channel), and permanent / volatile / bearer access keys enforced in the back end.

---

**Documentation & source**

github.com/kwirthmagnify/kwirth  Apache License 2.0  BYO-LLM AI · single-pod observability

REFERENCE

# Useful links

## Documentation

**Project website**
http://kwirthmagnify.dev

**Product guide (docsify)**
served in-app at /docs

## Install

**Releases & desktop installers**
github.com/kwirthmagnify/kwirth/releases

**Container image (Docker Hub)**
kwirthmagnify/kwirth

**Helm chart**
…/kwirth/tree/master/deploy/helm

**External CLI (npm)**
@kwirthmagnify/kwirth-external

## Source & community

**Repository**
github.com/kwirthmagnify/kwirth

**Issues**
github.com/kwirthmagnify/kwirth/issues

**Discussions**
github.com/kwirthmagnify/kwirth/discussions

**License — Apache 2.0**
…/kwirth/blob/master/LICENSE

# kwirth

## Thank you

Kubernetes observability & operations — in a single pod.

---

github.com/kwirthmagnify/kwirth · Apache License 2.0